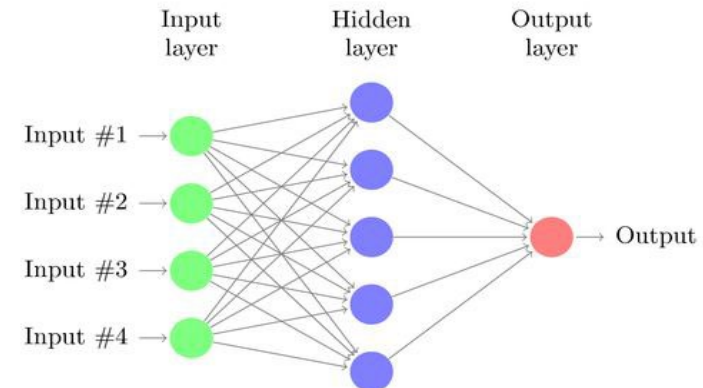
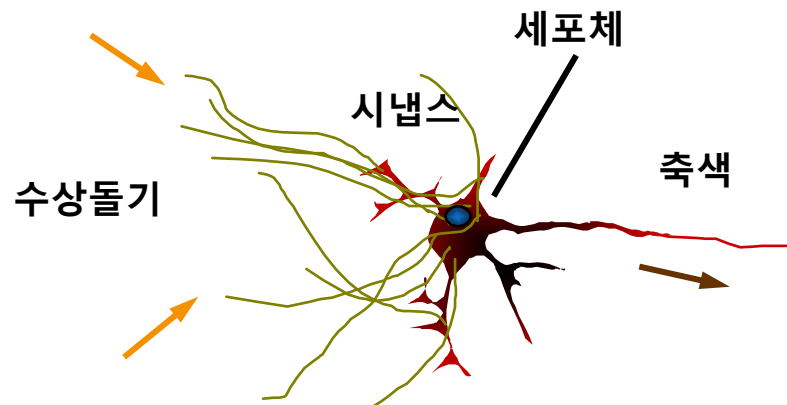


6강 부록: SVM과 Neural Network

서울대학교 통계학과
김용대

Neural Network Architecture

- 인공 신경망 (Artificial Neural Network)
 - 기계 학습 (Machine Learning)의 한 분야로, 생물학의 뇌 구조 (신경망)를 모방하여 만든 수학적 모형
 - 인간의 뇌가 문제를 해결하는 방식과 유사하게 구현
 - 인공 신경망의 역사
 - 1943년, W.S.McCulloch와 W.Pitts의 MP-model
 - 1957년, F.Rosenblatt의 single layer perceptron
 - 1986년, D.E.Rumelhart의 multilayer perceptrons
 - 2006년, G.E.Hinton의 Deep Neural Network에서의 혁신적인 학습 방법 개발



Neural Network Architecture

- 인공 신경망의 함수구조: 1개의 중간층 (Hidden layer)을 갖는 Neural Network 모형
 - K개의 output node를 갖는 neural network:

$$z_m^{(0)} = b_m^{(0)} + \mathbf{x}^T \mathbf{w}_m^{(0)}, \quad h_m = \sigma(z_m^{(0)}), \quad m = 1, \dots, M$$

$$z_k^{(1)} = b_k^{(1)} + \mathbf{h}^T \mathbf{w}_k^{(1)}, \quad f_k(\mathbf{x}) = g_k(\mathbf{z}^{(1)}), \quad k = 1, \dots, K$$

여기서 $\mathbf{x} = (x_1, \dots, x_p)^T$, $\mathbf{h} = (h_1, \dots, h_M)^T$, $\mathbf{z}^{(1)} = (z_1^{(1)}, \dots, z_K^{(1)})^T$ 이다.

- $\sigma(\cdot)$: 활성화함수 (activation function), 흔히 sigmoid 또는 tanh 함수를 사용

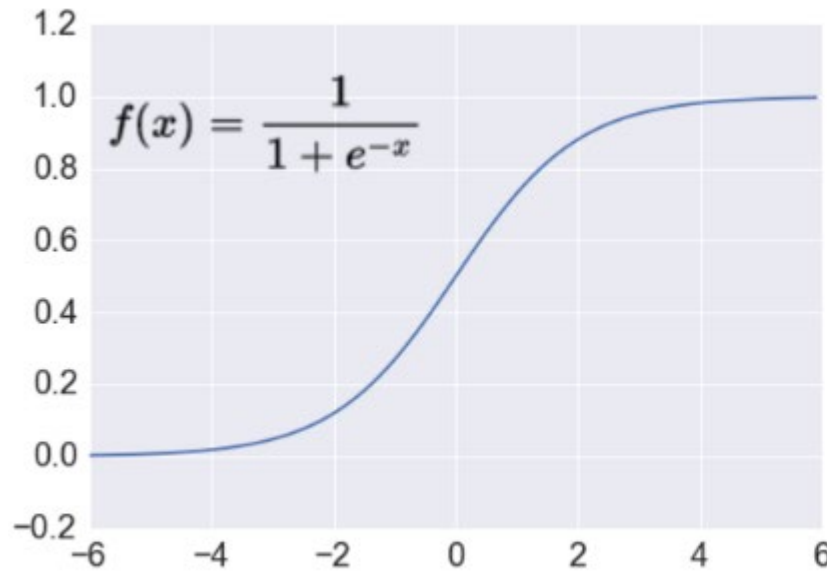
$$\sigma_{sig}(v) = 1/(1 + e^{-v}), \quad \sigma_{tanh}(v) = (e^v - e^{-v})/(e^v + e^{-v})$$

- $g_k(t)$: 출력함수 (output function)
- 회귀분석: $g_k(t) = t$
- 분류문제: softmax 함수를 사용 ($g_k(t) = e^{t_k} / \sum_{l=1}^K e^{t_l}$)

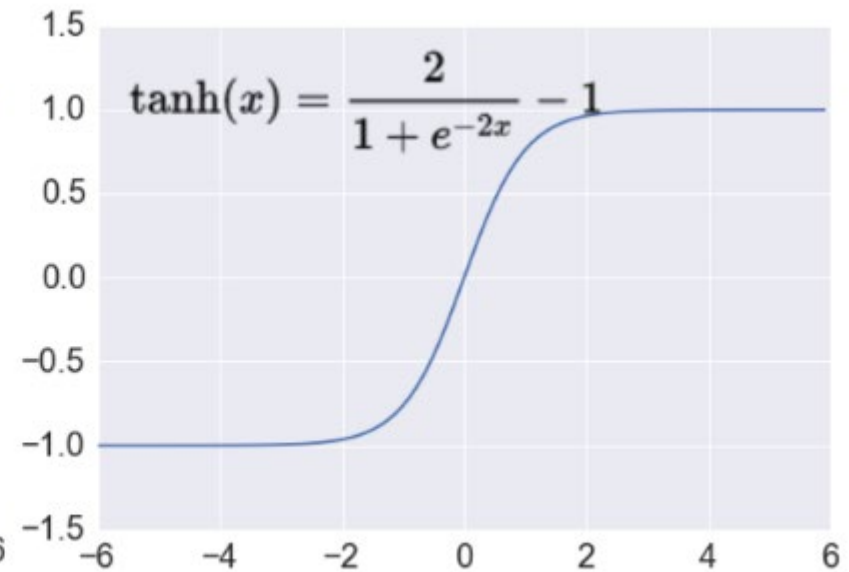
Neural Network Architecture

- 인공 신경망의 함수구조 – Activation function

Sigmoid

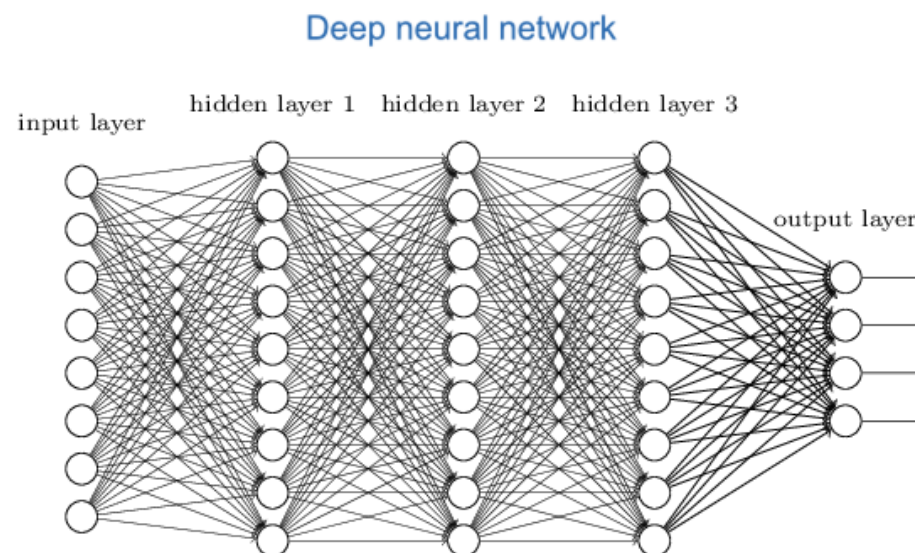
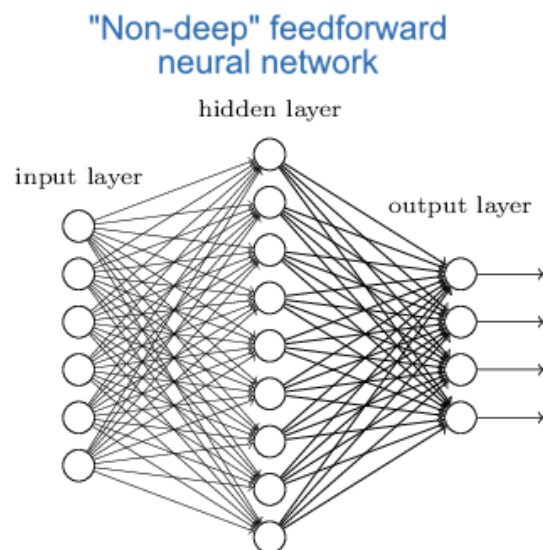


Tanh



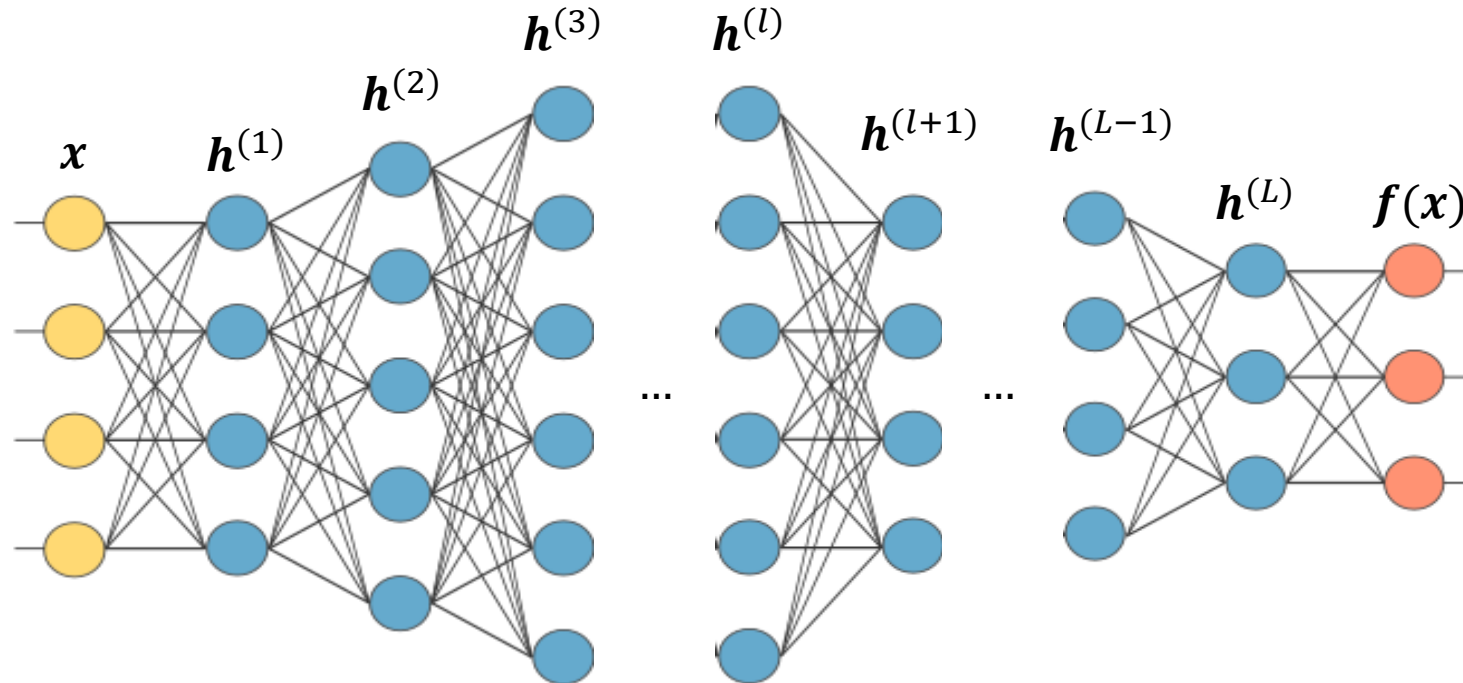
Neural Network Architecture

- 딥러닝이란?
 - 2개 이상의 중간층을 가지고 있는 신경망 모형



Neural Network Architecture

- Fully connected NN의 함수구조 : L 개의 중간층 (Hidden layer)을 갖는 Neural Network 모형
 - $\mathbf{h}^{(l)}$, $l = 0, \dots, L + 1$ 을 l 번째 중간층의 노드 값이라고 하자. ($\mathbf{h}^{(0)} = \mathbf{x}, \mathbf{h}^{(L+1)} = f(\mathbf{x})$)



Neural Network Architecture

- Fully connected NN의 함수구조 : L 개의 중간층 (Hidden layer)을 갖는 Neural Network 모형
 - $\mathbf{h}^{(l)}$, $l = 0, \dots, L + 1$ 을 l 번째 중간층의 노드 값이라고 하자. ($\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L+1)} = f(\mathbf{x})$)

$$z_j^{(l+1)} = b_j^{(l)} + \sum_{i=1}^{n_l} w_{ij}^{(l)} h_i^{(l)}, \quad l = 0, \dots, L$$

$$h_j^{(l+1)} = \sigma(z_j^{(l+1)}), \quad l = 0, \dots, L - 1 \quad : \text{hidden layer}$$

$$f_j(\mathbf{x}) = h_j^{(L+1)} = g(z_j^{(L+1)}) \quad : \text{output layer}$$

Neural Network 흥망성쇠

- 2006년 이전의 DNN의 문제들
 - Vanishing gradients problem
 - Vanishing gradients problem으로 인해 Deep Neural Network에서 역전파 알고리즘이 좋지 않은 추정값을 제공 (local minima).
 - Deep Neural Network가 한 개의 중간층을 갖는 Neural Network보다 더 나쁜 성능을 나타내는 경우가 생김.
 - Running time
 - Deep Neural Network은 대개 한 개의 중간층을 갖는 Neural Network보다 더 많은 수의 모수를 필요로 함.
 - 더 많은 수의 모수를 추정하기 위해 더 많은 시간 소요.



은닉층이 하나인 Neural Network가 주로 사용됨

Neural Network 흥망성쇠

- 2006년 이전의 DNN의 문제들
 - Shallow Neural Network as an Universal Approximator

G.Cybenko (1989)

Theorem 2. *Let σ be any continuous sigmoidal function. Then finite sums of the form*

$$G(x) = \sum_{j=1}^N \alpha_j \sigma(y_j^T x + \theta_j)$$

are dense in $C(I_n)$. In other words, given any $f \in C(I_n)$ and $\varepsilon > 0$, there is a sum, $G(x)$, of the above form, for which

$$|G(x) - f(x)| < \varepsilon \quad \text{for all } x \in I_n.$$

where sigmoidal(in this theorem) means :

$$\sigma(t) \rightarrow \begin{cases} 1 & \text{as } t \rightarrow \infty \\ 0 & \text{as } t \rightarrow -\infty \end{cases}$$

Neural Network 흥망성쇠

- 2006년 이전의 DNN의 문제들

- NN vs SVM

- Recall that the neural network kernel is

$$K(\mathbf{x}_1, \mathbf{x}_2) = \tanh(\kappa_1 \mathbf{x}_1^T \mathbf{x}_2 + \kappa_2).$$

- Suppose $\kappa_1 = 1$ and $\kappa_2 = 0$.
 - The estimator of the SVM (or kernel machine) is given as

$$\hat{f}(\mathbf{x}) = \beta_0 + \sum_{i=1}^n \beta_i \tanh(\mathbf{x}_i^T \mathbf{x})$$

- Compare this with the one-hidden layer neural network

$$\hat{f}(\mathbf{x}) = \beta_0 + \sum_{k=1}^M \beta_k \tanh(\alpha_k^T \mathbf{x}).$$

- Note that the SVM is a convex problem while NN is not.

→ 많은 학자들이 1990년대부터 2000년대 초까지 Deep Neural Network에 관심을 보이지 않게 됨.

Neural Network 흥망성쇠

- Pre-training
 - 2006년 G.E.Hinton 교수가 처음으로 제안
 - 역전파 알고리즘의 문제
 - 결과가 해의 초기값에 의존
 - 좋은 해의 초기값을 찾기 어려움
- Hinton의 아이디어
 - 입력변수만 사용하여 모수 추정 (Pre-training)
 - 추정된 모수를 해의 초기값으로 사용
- 딥러닝이 급격하게 발전하게 되는 결정적인 계기가 됨.

A fast learning algorithm for deep belief nets *

Geoffrey E. Hinton and Simon Osindero
Department of Computer Science University of Toronto
10 Kings College Road
Toronto, Canada M5S 3G4
{hinton, osindero}@cs.toronto.edu

Yee-Whye Teh
Department of Computer Science
National University of Singapore
3 Science Drive 3, Singapore, 117543
tehyw@comp.nus.edu.sg

ture bars in Fig. 1 summarize the measured SHG signals. For excitation of the LC resonance in Fig. 1A (horizontal incident polarization), we find an SHG signal that is 500 times above the noise level. As expected for SHG, this signal closely scales with the square of the incident power (Fig. 2A). The polarization of the SHG emission is nearly vertical (Fig. 2B). The small angle with respect to the vertical is due to deviations from perfect mirror symmetry of the SRRs (see electron micrographs in Fig. 1). Small detuning of the LC resonance toward smaller wavelength (i.e., to 1.3- μ m wavelength) reduces the SHG signal strength from 100% to 20%. For excitation of the Mie resonance with vertical incident polarization in Fig. 1D, we find a small signal just above the noise level. For excitation of the Mie resonance with horizontal incident polarization in Fig. 1C, a small but significant SHG emission is found, which is again po-

Reducing the Dimensionality of Data with Neural Networks

G. E. Hinton* and R. R. Salakhutdinov

High-dimensional data can be converted to low-dimensional codes by training a multilayer neural network with a small central layer to reconstruct high-dimensional input vectors. Gradient descent can be used for fine-tuning the weights in such "autoencoder" networks, but this works well only if the initial weights are close to a good solution. We describe an effective way of initializing the weights that allows deep autoencoder networks to learn low-dimensional codes that work much better than principal components analysis as a tool to reduce the dimensionality of data.

Dimensionality reduction facilitates the classification, visualization, communication, and storage of high-dimensional data. A simple and widely used method is principal components analysis (PCA), which finds the directions of greatest variance in the data set and represents each data point by its coordinates along each of these directions. We describe a nonlinear generalization of PCA that uses an adaptive, multilayer "encoder" network

Thank You

